

MACHINE LEARNING + STATISTICS REPORT

Predicting Undernourishment in Africa

Ten machine-learning algorithms, validated honestly, and supported by statistical models

Kingsley Amegah

Health Data Scientist

Python · scikit-learn · statsmodels · SHAP | World Bank Open Data | github.com/Kingsley-amg/africa-nutrition-ml

Executive summary

This project predicts **undernourishment** (the prevalence of food insecurity) across **49 African countries** over 2001-2023 (1,094 country-year observations), using World Bank data. It compares ten machine-learning algorithms and supports the findings with formal statistical models.

- **The headline is a lesson in honest evaluation.** Under ordinary random cross-validation the best model reaches R-squared of 0.98, which looks superb - but it is misleading, because the same country appears in both training and test data.
- Under **country-held-out validation**, the best model (Extra Trees) reaches only R-squared 0.17: predicting undernourishment for a country never seen in training is genuinely hard.
- **Income is the dominant correlate** of undernourishment, followed by water access, sanitation and health spending; higher fertility and agricultural employment go with more undernourishment.
- These directions are **consistent across correlation, OLS regression and a mixed-effects panel model**, which together explain why prediction is hard: much of the variation is persistent and country-specific.

1. Introduction and question

Undernourishment remains a defining development challenge across much of Africa. This project asks two linked questions: **how well can we predict** a country's undernourishment from its socio-economic, agricultural and health conditions, and **which factors are most associated** with it? The first is a machine-learning question, the second a statistical one, and the project treats them together.

2. Data and methods

Data come from the **World Bank Open Data API**: the prevalence of undernourishment as the outcome, plus twelve determinants (GDP per capita, health spending, water and sanitation access, fertility, rural population share, food production, agricultural employment and value added, population growth, primary enrolment and inflation). GDP and health spending were log-transformed. Ten regression algorithms were compared under two cross-validation designs - ordinary random K-fold and **GroupKFold grouped by country** - and the findings were checked with correlation analysis, an OLS regression with full inference, and a mixed-effects panel model. Full code is in Appendices A and B.

3. Exploratory analysis

Undernourishment has fallen on average but remains high and uneven across African regions (Figure 1). The strongest bivariate correlates are water access, income and sanitation (all negative) and fertility (positive).

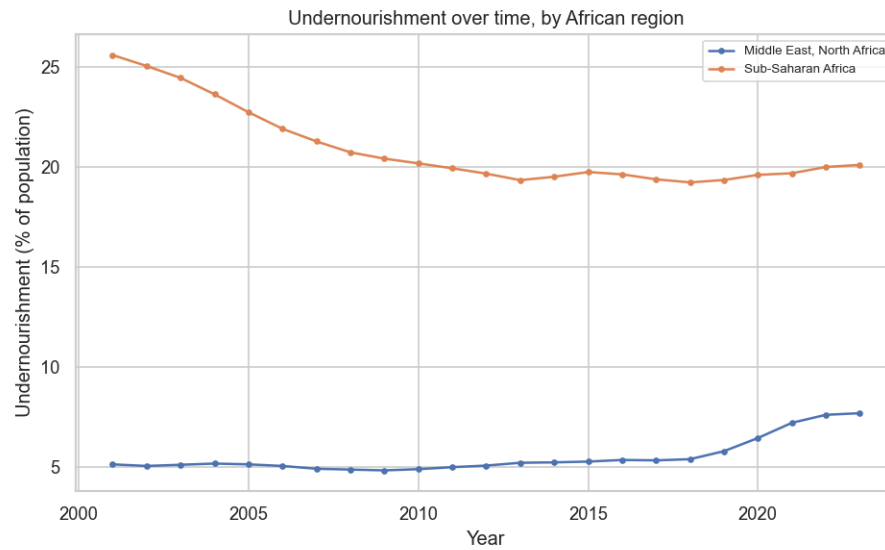


Figure 1. Mean undernourishment over time, by African region.

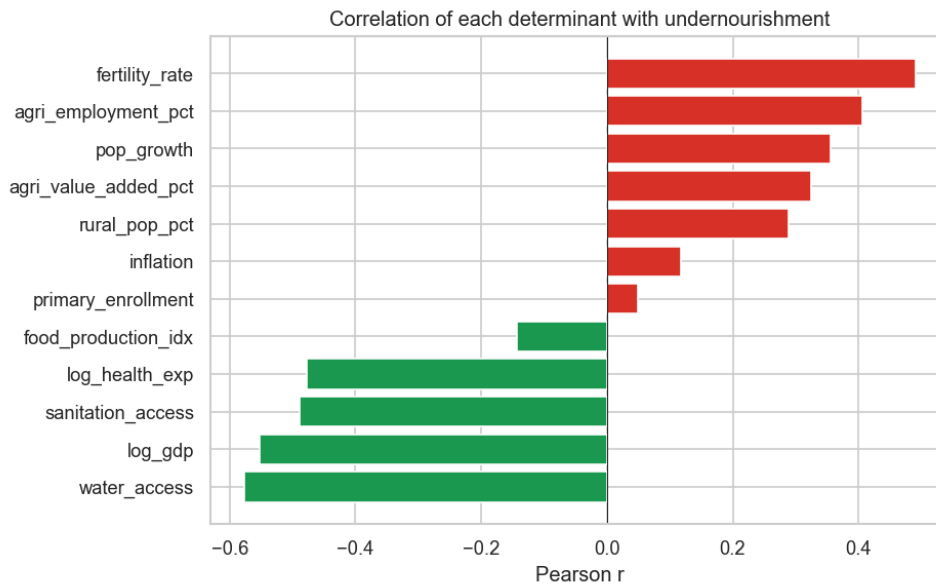


Figure 2. Correlation of each determinant with undernourishment.

4. Machine learning: ten algorithms, two validation designs

Algorithm	Random-CV R2	Grouped-CV R2	RMSE	MAE
Extra Trees	0.98	0.17	10.7	8.2
Random Forest	0.94	0.10	10.9	8.5
Gradient Boosting	0.87	0.10	11.1	8.5
ElasticNet	0.42	0.04	11.4	8.7
Lasso	0.42	0.01	11.5	8.9

Ridge	0.42	-0.04	11.8	9.2
Linear Regression	0.42	-0.04	11.8	9.2
K-Nearest Neighbors	0.83	-0.07	12.2	9.1
SVR (RBF)	0.76	-0.15	12.5	9.6
Decision Tree	0.76	-0.35	13.1	10.3

The contrast between the two columns is the most important result. Random cross-validation rewards models that memorise each country's level from its other years, inflating R-squared to 0.98. The honest test - holding out whole countries - collapses this to 0.17 for the best model, and to around zero or below for the linear and simpler models. Tree ensembles (Extra Trees, Random Forest) generalise best, but even they explain only a modest share of variation in unseen countries.

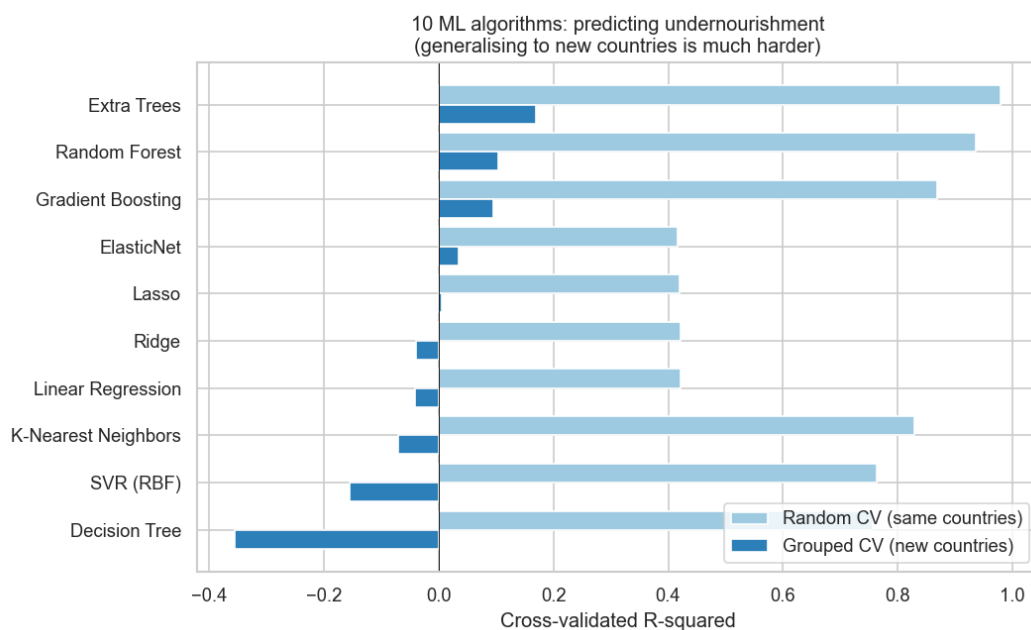


Figure 3. Ten algorithms under random vs. country-grouped cross-validation.

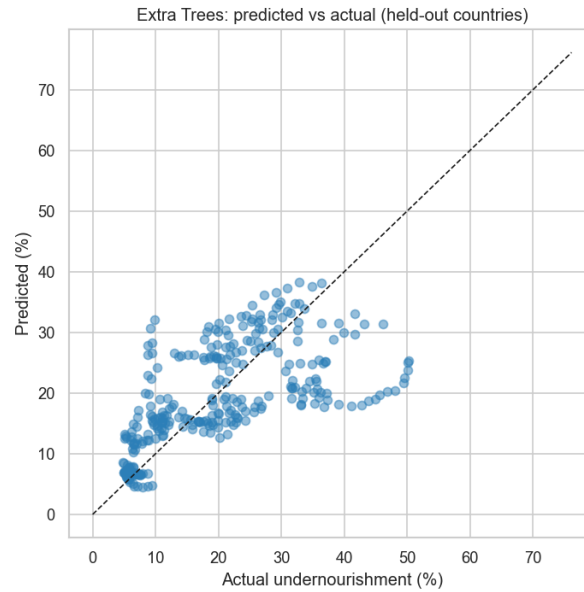


Figure 4. Best model: predicted vs. actual undernourishment for held-out countries.

5. Explainability (SHAP)

SHAP values for the random-forest model confirm the statistical picture: low income, low water and sanitation access, and high fertility push predicted undernourishment up. The agreement between a flexible ML model and the regression models is reassuring - the signal is real, not an artefact of one method.

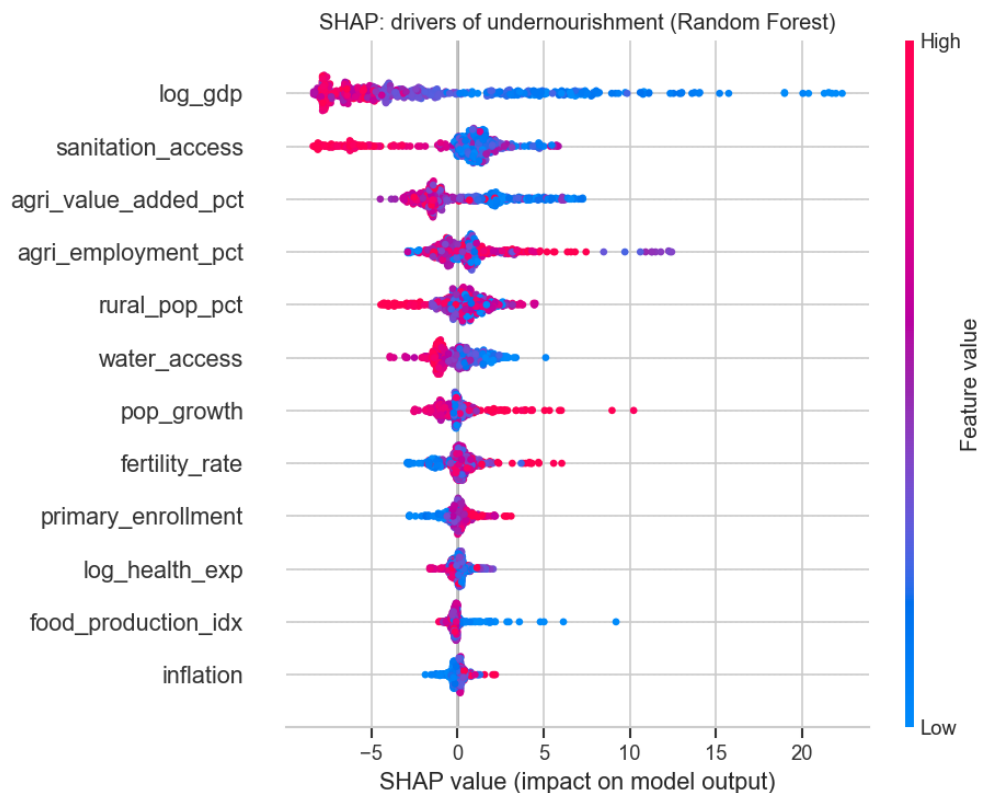


Figure 5. SHAP summary: how each determinant moves predicted undernourishment.

6. Statistical models

A standardised OLS regression explains 45% of the in-sample variation. Coefficients are reported with 95% confidence intervals (Figure 6); income has by far the largest standardised effect. Multicollinearity is moderate (max VIF = 6.9), so individual coefficients - particularly for the correlated income and spending variables - should be read with care. Because the data is a country panel with repeated measures, a **mixed-effects model with a random intercept per country** was also fitted; it confirms income as a strong negative correlate while absorbing persistent country differences, which is precisely why purely cross-country prediction is limited.

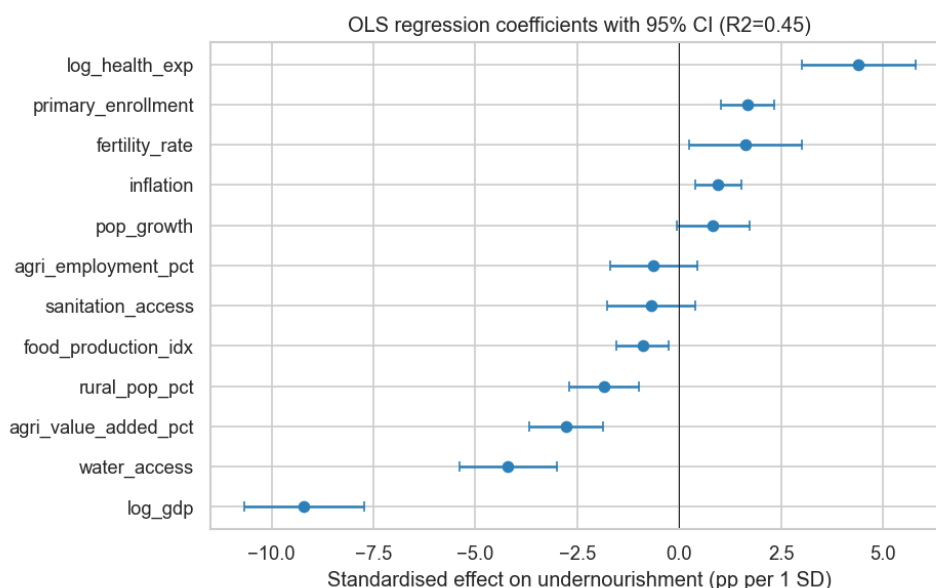


Figure 6. Standardised OLS coefficients with 95% confidence intervals.

7. Conclusion

Across ten algorithms and three statistical models, the determinants of undernourishment in Africa are consistent and intuitive: income, clean water, sanitation and lower fertility are associated with less food insecurity. Yet the project's most valuable lesson is methodological. A model that looks near-perfect under careless validation is shown, under honest country-held-out testing, to generalise only modestly - because undernourishment is deeply country-specific. Reporting both is what separates a trustworthy analysis from an over-optimistic one.

8. Limitations

- Country-level (ecological) analysis; relationships need not hold for individuals.
- Observational data: associations, not causal effects.
- Some determinants are collinear (max VIF near 7), affecting individual OLS coefficients.

- Undernourishment is partly modelled by the World Bank and updated on a lag.
- Survey-based predictors carry measurement error and uneven coverage.

Appendix A - Data extraction (01_extract.py)

```
"""
Step 1 - Build the Africa nutrition panel from the World Bank API.
Target: prevalence of undernourishment (% of population).
Predictors: socio-economic, agricultural, health and demographic determinants.
Country-year panel for African countries, 2000-2024. No API key needed.

Run: python 01_extract.py -> data/africa_nutrition.csv
"""
import time
from functools import reduce
from pathlib import Path
import requests
import pandas as pd

BASE = "https://api.worldbank.org/v2"
DATA = Path("data"); DATA.mkdir(exist_ok=True)
YEARS = "2000:2024"

TARGET = ("SN.ITK.DEFC.ZS", "undernourishment") # % of population (the outcome)

PREDICTORS = {
    "NY.GDP.PCAP.PP.KD": "gdp_per_capita", # PPP, constant intl $
    "SH.H2O.BASW.ZS": "water_access", # % basic drinking water
    "SH.STA.BASS.ZS": "sanitation_access", # % basic sanitation
    "SP.DYN.TFRT.IN": "fertility_rate", # births per woman
    "SP.RUR.TOTL.ZS": "rural_pop_pct", # % living rurally
    "AG.PRD.FOOD.XD": "food_production_idx", # food production index
    "SL.AGR.EMPL.ZS": "agri_employment_pct", # % employed in agriculture
    "NV.AGR.TOTL.ZS": "agri_value_added_pct", # agriculture, % of GDP
    "SP.POP.GROW": "pop_growth", # annual population growth %
    "SH.XPD.CHEX.PC.CD": "health_exp_per_capita", # current US$
    "SE.PRM.ENRR": "primary_enrollment", # gross primary enrolment %
    "FP.CPI.TOTL.ZG": "inflation", # consumer price inflation %
}

def fetch(code, name):
    url = f"{BASE}/country/all/indicator/{code}"
    for _ in range(3):
        r = requests.get(url, params={"format": "json", "date": YEARS, "per_page": 20000}, timeout=60)
        j = r.json()
        if r.ok and isinstance(j, list) and len(j) > 1 and j[1]:
            df = pd.DataFrame([{"iso3": d["countryiso3code"], "year": int(d["date"]),
                               name: d["value"]} for d in j[1]])
            df = df[df["iso3"].str.len() == 3].dropna(subset=[name])
            return df.drop_duplicates(["iso3", "year"])
        time.sleep(2)
    raise RuntimeError(f"failed {code}")

print("downloading target + predictors ...")
frames = [fetch(*TARGET)] + [fetch(c, n) for c, n in PREDICTORS.items()]
panel = reduce(lambda l, r: l.merge(r, on=["iso3", "year"], how="outer"), frames)

# Country metadata; keep African countries only
meta = requests.get(f"{BASE}/country", params={"format": "json", "per_page": 400}, timeout=60).json()[1]
md = pd.DataFrame([{"iso3": c["id"], "country": c["name"].strip(),
                    "region": c["region"]["value"].strip(),
                    "income_group": c["incomeLevel"]["value"].strip()} for c in meta])
african = md[md.region.isin(["Sub-Saharan Africa",
                             "Middle East, North Africa, Afghanistan & Pakistan"])]

# restrict the MENA bucket to actual African states
north_africa = {"DZA", "EGY", "LBY", "MAR", "TUN", "SDN", "ESH"}
african = african[(african.region == "Sub-Saharan Africa") | (african.iso3.isin(north_africa))]

panel = panel.merge(african, on="iso3", how="inner").sort_values(["country", "year"])
panel = panel.dropna(subset=["undernourishment"]) # need the target

panel.to_csv(DATA / "africa_nutrition.csv", index=False)
print("\n=== EXTRACT COMPLETE ===")
print(f"Countries: {panel.iso3.nunique()} | years: {panel.year.min()}-{panel.year.max()}")
print(f"Rows (with target): {len(panel):,}")
```



```
print("Predictor coverage (% non-missing):")
for n in PREDICTORS.values():
    print(f" {n:22} {panel[n].notna().mean():.0%}")
```

Appendix B - Analysis (02_analyze.py)

```
"""
Step 2 - Predict undernourishment in Africa with 10 ML algorithms, and support
the findings with classical statistical models.

ML: 10 regression algorithms compared with GroupKFold cross-validation
    (grouped by country, so we test generalisation to UNSEEN countries).
STATS: correlation analysis, OLS multiple regression (CIs, p-values, VIF,
    diagnostics), and a mixed-effects panel model (random intercept by country).
INTERP: SHAP on the best tree model.

Run: python 02_analyze.py -> figures + metrics.json in ./outputs
"""
import json, warnings
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr
import statsmodels.api as sm
from statsmodels.regression.mixed_linear_model import MixedLM
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.model_selection import GroupKFold, KFold, cross_validate, GroupShuffleSplit
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (RandomForestRegressor, GradientBoostingRegressor,
                             ExtraTreesRegressor, AdaBoostRegressor)
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
OUT = Path("outputs"); OUT.mkdir(exist_ok=True)
RS = 42
M = {}

df = pd.read_csv("data/africa_nutrition.csv")
# Log-transform skewed monetary variables
df["log_gdp"] = np.log10(df["gdp_per_capita"])
df["log_health_exp"] = np.log10(df["health_exp_per_capita"])

FEATURES = ["log_gdp", "log_health_exp", "water_access", "sanitation_access",
            "fertility_rate", "rural_pop_pct", "food_production_idx",
            "agri_employment_pct", "agri_value_added_pct", "pop_growth",
            "primary_enrollment", "inflation"]
TARGET = "undernourishment"

d = df.dropna(subset=[TARGET]).copy()
X = d[FEATURES]; y = d[TARGET]; groups = d["iso3"]
M["n_rows"] = int(len(d)); M["n_countries"] = int(d.iso3.nunique())
M["year_range"] = [int(d.year.min()), int(d.year.max())]
M["target_mean"] = round(float(y.mean()), 1)

# -----
# 0. EDA: undernourishment over time by region
# -----
reg = d.groupby(["region", "year"])[TARGET].mean().reset_index()
plt.figure(figsize=(8, 5))
for r_, s in reg.groupby("region"):
    plt.plot(s.year, s[TARGET], marker="o", ms=3, label=r_.replace("Afghanistan & Pakistan", ""))
plt.title("Undernourishment over time, by African region"); plt.xlabel("Year")
plt.ylabel("Undernourishment (% of population)"); plt.legend(fontsize=8)
plt.tight_layout(); plt.savefig(OUT / "01_trend_by_region.png", dpi=120); plt.close()
```

```

# -----
# 1. Correlation analysis (with significance)
# -----
cors = []
for f in FEATURES:
    sub = d[[f, TARGET]].dropna()
    r, p = pearsonr(sub[f], sub[TARGET])
    cors.append((f, r, p))
cor_df = pd.DataFrame(cors, columns=["feature", "r", "p"]).sort_values("r")
M["correlations_with_target"] = {f: round(r, 2) for f, r, p in cors}
plt.figure(figsize=(8, 5))
plt.barh(cor_df.feature, cor_df.r, color=np.where(cor_df.r > 0, "#d73027", "#1a9850"))
plt.axvline(0, color="k", lw=0.6)
plt.title("Correlation of each determinant with undernourishment")
plt.xlabel("Pearson r"); plt.tight_layout()
plt.savefig(OUT / "02_correlations.png", dpi=120); plt.close()

# -----
# 2. TEN machine-learning algorithms (GroupKFold by country)
# -----
def pipe(model, scale):
    steps = [("imp", SimpleImputer(strategy="median"))]
    if scale: steps.append(("sc", StandardScaler()))
    steps.append(("m", model))
    return Pipeline(steps)

MODELS = {
    "Linear Regression": (LinearRegression(), True),
    "Ridge": (Ridge(alpha=1.0), True),
    "Lasso": (Lasso(alpha=0.1), True),
    "ElasticNet": (ElasticNet(alpha=0.1, l1_ratio=0.5), True),
    "K-Nearest Neighbors": (KNeighborsRegressor(n_neighbors=10), True),
    "SVR (RBF)": (SVR(C=10, gamma="scale"), True),
    "Decision Tree": (DecisionTreeRegressor(max_depth=6, random_state=RS), False),
    "Random Forest": (RandomForestRegressor(n_estimators=400, random_state=RS, n_jobs=-1), False),
    "Gradient Boosting": (GradientBoostingRegressor(random_state=RS), False),
    "Extra Trees": (ExtraTreesRegressor(n_estimators=400, random_state=RS, n_jobs=-1), False),
}

# Two validation designs:
# - random K-fold: rows shuffled (a country can appear in train AND test)
# - GroupKFold by country: whole countries held out -> tests generalisation
gkf = GroupKFold(n_splits=5)
kf = KFold(n_splits=5, shuffle=True, random_state=RS)
scoring = {"r2": "r2", "rmse": "neg_root_mean_squared_error", "mae": "neg_mean_absolute_error"}
rows = []
for name, (mdl, scale) in MODELS.items():
    cg = cross_validate(pipe(mdl, scale), X, y, groups=groups, cv=gkf, scoring=scoring, n_jobs=-1)
    cr = cross_validate(pipe(mdl, scale), X, y, cv=kf, scoring=scoring, n_jobs=-1)
    rows.append({"model": name,
                "R2_random": cr["test_r2"].mean(),
                "R2_grouped": cg["test_r2"].mean(),
                "RMSE_grouped": -cg["test_rmse"].mean(),
                "MAE_grouped": -cg["test_mae"].mean()})
board = pd.DataFrame(rows).sort_values("R2_grouped", ascending=False).reset_index(drop=True)
print(board.round(3).to_string(index=False))
M["leaderboard"] = [{"model": r.model, "R2_random": round(r.R2_random, 3),
                    "R2_grouped": round(r.R2_grouped, 3), "RMSE_grouped": round(r.RMSE_grouped, 2),
                    "MAE_grouped": round(r.MAE_grouped, 2)} for r in board.itertuples()]
M["best_model"] = board.iloc[0]["model"]
M["best_r2_grouped"] = round(float(board.iloc[0]["R2_grouped"]), 3)
M["best_r2_random"] = round(float(board["R2_random"].max()), 3)

ypos = np.arange(len(board))[:-1]
plt.figure(figsize=(9, 5.5))
plt.barh(ypos + 0.2, board.R2_random, height=0.4, color="#9ecae1", label="Random CV (same countries)")
plt.barh(ypos - 0.2, board.R2_grouped, height=0.4, color="#2c7fb8", label="Grouped CV (new countries)")
plt.yticks(ypos, board.model)
plt.axvline(0, color="k", lw=0.6)
plt.xlabel("Cross-validated R-squared"); plt.legend(loc="lower right")
plt.title("10 ML algorithms: predicting undernourishment\n(generalising to new countries is much harder)")
plt.tight_layout(); plt.savefig(OUT / "03_ml_leaderboard.png", dpi=120); plt.close()

# Predicted vs actual for the best model on a country-held-out test split

```

```

gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=RS)
tr, te = next(gss.split(X, y, groups))
best_name = board.iloc[0]["model"]; best_mdl, best_scale = MODELS[best_name]
best_pipe = pipe(best_mdl, best_scale).fit(X.iloc[tr], y.iloc[te])
pred = best_pipe.predict(X.iloc[te])
M["holdout_r2"] = round(float(r2_score(y.iloc[te], pred)), 3)
M["holdout_mae"] = round(float(mean_absolute_error(y.iloc[te], pred)), 2)
plt.figure(figsize=(6, 6))
plt.scatter(y.iloc[te], pred, alpha=0.5, color="#2c7fb8")
lims = [0, max(y.max(), pred.max()) * 1.05]
plt.plot(lims, lims, "k--", lw=1)
plt.xlabel("Actual undernourishment (%)"); plt.ylabel("Predicted (%)")
plt.title(f"{best_name}: predicted vs actual (held-out countries)")
plt.tight_layout(); plt.savefig(OUT / "04_pred_vs_actual.png", dpi=120); plt.close()

# -----
# 3. SHAP on a Random Forest (tree model -> reliable explanations)
# -----
import shap
rf = pipe(RandomForestRegressor(n_estimators=400, random_state=RS, n_jobs=-1), False).fit(X, y)
Ximp = pd.DataFrame(rf.named_steps["imp"].transform(X), columns=FEATURES)
expl = shap.TreeExplainer(rf.named_steps["m"])
sv = expl.shap_values(Ximp.sample(min(500, len(Ximp)), random_state=RS))
plt.figure()
shap.summary_plot(sv, Ximp.sample(min(500, len(Ximp)), random_state=RS), show=False)
plt.title("SHAP: drivers of undernourishment (Random Forest)")
plt.tight_layout(); plt.savefig(OUT / "05_shap.png", dpi=120, bbox_inches="tight"); plt.close()

# -----
# 4. Statistical models
# -----
# Build a standardised, median-imputed modelling matrix
stat = d[FEATURES + [TARGET, "iso3"]].copy()
for f in FEATURES:
    stat[f] = stat[f].fillna(stat[f].median())
Xz = (stat[FEATURES] - stat[FEATURES].mean()) / stat[FEATURES].std()

# 4a. OLS with inference + VIF
Xc = sm.add_constant(Xz)
ols = sm.OLS(stat[TARGET], Xc).fit()
M["ols_r2"] = round(float(ols.rsquared), 3)
M["ols_coefs"] = {k: round(float(v), 2) for k, v in ols.params.drop("const").items()}
vif = {f: round(float(variance_inflation_factor(Xz.values, i)), 1) for i, f in enumerate(FEATURES)}
M["max_vif"] = max(vif.values())

coefs = ols.params.drop("const").sort_values()
ci = ols.conf_int().drop("const")
plt.figure(figsize=(8, 5))
plt.errorbar(coefs.values, range(len(coefs)),
             xerr=[(coefs - ci[0][coefs.index]).values, (ci[1][coefs.index] - coefs).values],
             fmt="o", color="#2c7fb8", capsize=3)
plt.xticks(range(len(coefs)), coefs.index)
plt.axvline(0, color="k", lw=0.6)
plt.xlabel("Standardised effect on undernourishment (pp per 1 SD)")
plt.title(f"OLS regression coefficients with 95% CI (R2={ols.rsquared:.2f})")
plt.tight_layout(); plt.savefig(OUT / "06_ols_coefficients.png", dpi=120); plt.close()

# 4b. Mixed-effects panel model: random intercept by country
mix = MixedLM(stat[TARGET], Xc, groups=stat["iso3"]).fit()
M["mixed_coefs"] = {k: round(float(v), 2) for k, v in mix.params.items()
                    if k not in ("const", "Group Var")}

with open(OUT / "metrics.json", "w") as f:
    json.dump(M, f, indent=2)
print("\nBest:", M["best_model"], "| grouped R2:", M["best_r2_grouped"],
      "| random R2:", M["best_r2_random"], "| OLS R2:", M["ols_r2"], "| max VIF:", M["max_vif"])
print("Done. Figures + metrics.json in ./outputs")

```